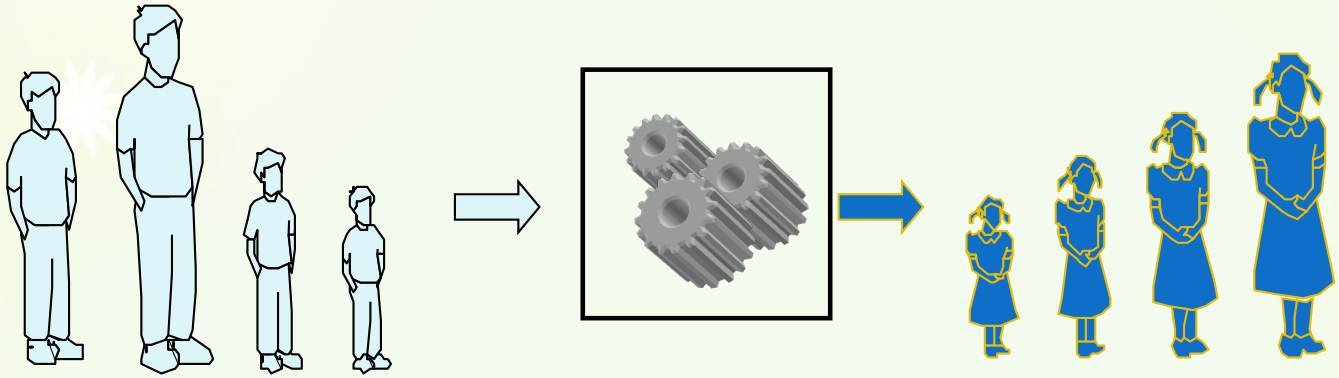
資料結構與演算法 I
(Data Structure and Algorithms I)
2025 fall

2025/09/12

Analysis of Algorithms

Analysis of Algorithms



Input **Algorithm** **Output**

An **algorithm** is a step-by-step procedure for solving a problem in a finite amount of time.

What is data structure

- An organization of information, usually in memory, for better algorithm efficiency.
- Or, a way to store and organize data in order to facilitate access and modifications.

$$2n^5-3n^2+11n+27$$

0	1	2	3	4	5	6
27	11	-3	0	0	2	0

- Try to write a program to calculate:

$$1+2+3+\ldots.100=?$$

Please take your pen and paper to write this program!

This is an **algorithm**

```
int i; int sum=0; int n=100;
for(i=1; i<=n; i++)
{
sum=sum+i;
}
cout<< “The total sum=”<<sum<<endl;
```

Is it right??? how about the efficiency?

Carolus Fridericus Gauss

sum= 1 + 2 + 3 + 4 + 5 +……+ 99 + 100
sum=100 + 99 +……+ 2 + 1
2*sum=101 + 101+…… +101+ 101

100個101

sum=(100*101)/2=5050

```
int sum=0; int n=100;
sum=n*(n+1)/2;
cout<<“The total sum=”<<sum<<endl;
```

當n趨近於一億或更多時，人腦應該可以算的比電腦快

Algorithm Specification

演算法是解決特定問題以求解步驟的描述，在電腦中為指令的有限序列，且每條指令表示一個或多個操作

- Definition:
  - An algorithm is a finite set of instructions that accomplishes a particular task and satisfy the following criteria:
    - Input: Zero or more quantities are externally supplied.
      - cout<<“Hello world”<<endl;
    - Output: At least one quantity is produced.
    - Definiteness: Each instruction is clear and unambiguous

確定性

    - Finiteness: Terminates after a number of steps.

有限性

    - Effectiveness: Every instruction must be basic enough to be carried out by a person using only pencil and paper. (表示演算法可以變成程式碼)

可行性

Request for the design of algorithm

- 正確性
  - Level 1: no error 只要沒錯誤就好
  - Level 2: Reasonable input→ satisfied output 合理的輸入
  - Level 3: Unreasonable input→ satisfied output 不合理的輸入
  - Level 4: Any input→ satisfied output 隨便怎樣輸入都會跑
- 可讀性
  - 便於閱讀、理解與交流
- 健壯性
  - 當輸入不合理時，演算法也能做出合理的處理方式，而非單純產生異常或莫名其妙的結果
- 高效率及低存取量
  - 效率：演算法執行時間
  - 存取量需求：演算法執行時所佔用的記憶體或外部硬碟存取空間

Pseudocode

- How do we describe an algorithm?
  - Human language (English, Chinese…)
  - Programming language
  - A mix of the above

```
程式開始
輸入 定價 與 數量
若 數量 >=5 則
    若 數量 >=10 則
        總價 = 定價 * 數量 * 0.75
    否則
        總價 = 定價 * 數量 * 0.8
    結束判斷
否則
        總價 = 定價 * 數量 * 0.9
結束判斷
輸出 總價
程式結束
```

Pseudocode

- High-level description of an algorithm
- More structured than English prose
  - 比直接語言描述有結構
- Less detailed than a program
- Preferred notation for describing algorithms
  - 符號描述
- Hides program design issues

Example: find max element of an array

```
Algorithm arrayMax(A, n)
    Input array A of n integers
    Output maximum element of A

    currentMax ← A[0]
    for i ← 1 to n – 1 do
        if A[i] > currentMax then
            currentMax ← A[i]
    return currentMax
```

Pseudocode Details

- Control flow
  - **if** … **then** … [**else** …]
  - **while** … **do** …
  - **repeat** … **until** …
  - **for** … **do** …
  - Indentation replaces braces
- Method declaration

  Algorithm ***method*** (***arg*** [, ***arg***…])

  Input …

  Output …
- Method/Function call

  var.method (***arg*** [, ***arg***…])
- Return value

  return ***expression***
- Expressions

  ← Assignment (like = in C++)

  = Equality testing (like == in C++)

  n^2 Superscripts and other mathematical formatting allowed

引數
變數

Selection Sort(1)

- Algorithm vs. Program
  - In computational theory, a program does not have to satisfy the finiteness. Why??
  - OS idle (除非當機或重新開關機，程式不會停止)
- Example 1.2 [Selection sort] Sort a collection of n >=1 integers.
  - From those integers that are currently unsorted, find the smallest and place it next in the sorted list.

Program 1.7 Selection sort algorithm

```
for (i = 0; i < n ; i++) {
      Examine a[i] to a[n-1] and suppose the smallest integer is at a[j];
      Interchange a[i] and a[j] ;
      }
```

Selection Sort(2)

After sort: a[0]<=a[1]<=a[2].....<=a[n-1]

```
void SelectionSort (int *a, int n)
{
    //sort the n integers a[0] to a[n-1] into nondecreasing order
    for (int i = 0; i < n ; i++ ) {
        int j=i;
        /* find smallest integer in a[i] to a[n-1] */
        for (int k = i+1 ; k< n ; k++)
                if (a[k] < a[j]) j = k;
        swap(a[i], a[j]);
    }
}
```

for loop 若無{ }
則僅執行一行程式

Search

- Sequence Search
  - 一個一個比對，最多要做n次(在n陣列中搜尋)
- Binary Search(要先排序)
  - 速度比較快，$2^m >= n$，所以最多做m次(在n陣列中搜尋)

Binary Search

- Example 1.3 [Binary search] Assume that we have n>=1 distinct integers that are already sorted in the array. Our task is to determine if the integer x is present and if so to return *j* such that x=a[*j*];otherwise return -1

Program 1.9 Algorithm for binary search

```
int BinarySearch(int *a, const int x, const int n)
{
  //Search the sorted array a[0], …a[n-1] for x
 Initialize left and right;
while (there are more elements)
    {
      Let middle be the middle element;
      if (x<a[middle]) set right to middle-1
      else if (x>a[middle]) set left to middle+1;
      else return middle;
    }
     Not found;
}
```

Program 1.10 C++ function for binary search

```
int BinarySearch( int *a, const int x, const int n)
{ //Search the sorted array a[0],….a[n-1] for x
        int left =0; right=n-1;
        while( left <=right ) {
                int middle=(left+right)/2;
                if(x<a[middle]) right=,middle-1;
                 else if(x>a[middle]) left=middle+1;
          else return middle;
                }//end of while
      return -1;//not found
}
```

Recursive Algorithms

- Recursive functions
- Direct recursion 直接遞迴
  - Functions call themselves before they are done.

A()→A()

- Indirect recursion 間接遞迴
  - Functions call other functions that again invoke the calling function.

A()→B()→A()

- Terminating condition: When satisfied, the function directly computes the output without calling itself.

For example:

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^{n-k} y^k$$

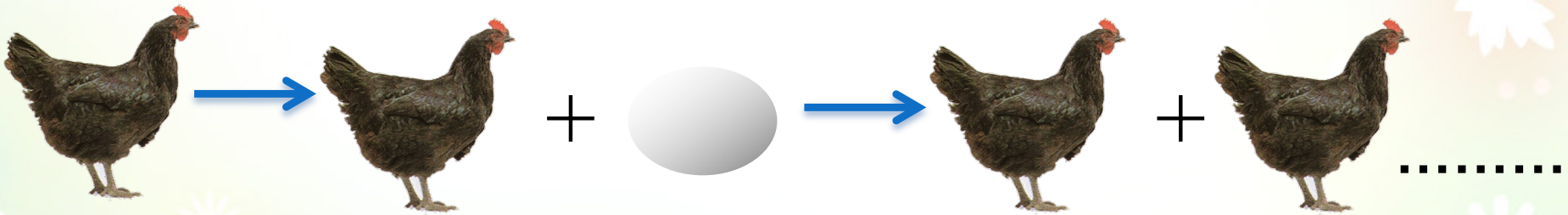
$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!} \quad \text{for} \quad 0 \le k \le n. \Rightarrow \binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \quad \forall n, k \in \mathbb{N}$$

二項式定理

Hen problem

- At the beginning of Day 1, there is a hen.
- Each hen lays an egg every 24 hours.
- Each egg takes 24 hours to become a hen.
- $F(n)$ = the number of hens at the end of day n.
- Give an algorithm to compute $F(n)$.

Fibonacci number



Day 1 Day 2 Day 3

- $F(n)$ = the number of hens at the end of day n.
- $F(1)=1$
  - (這一次雞的數量將會成為下一次蛋的數量，而會成為下下次新增的雞數量)
- F(2)=1
- F(3)=2
  - (上上次的雞數量→上次蛋數量→這次新增的雞;所以本次雞的數量=上次雞的數量 + 上上次雞的數量！)
- F(4)=3
- F(5)=5

F(n)=F(n-1)+E(n-1)=F(n-1)+F(n-2), n>2
Terminating condition: F(1)=1;F(2)=1

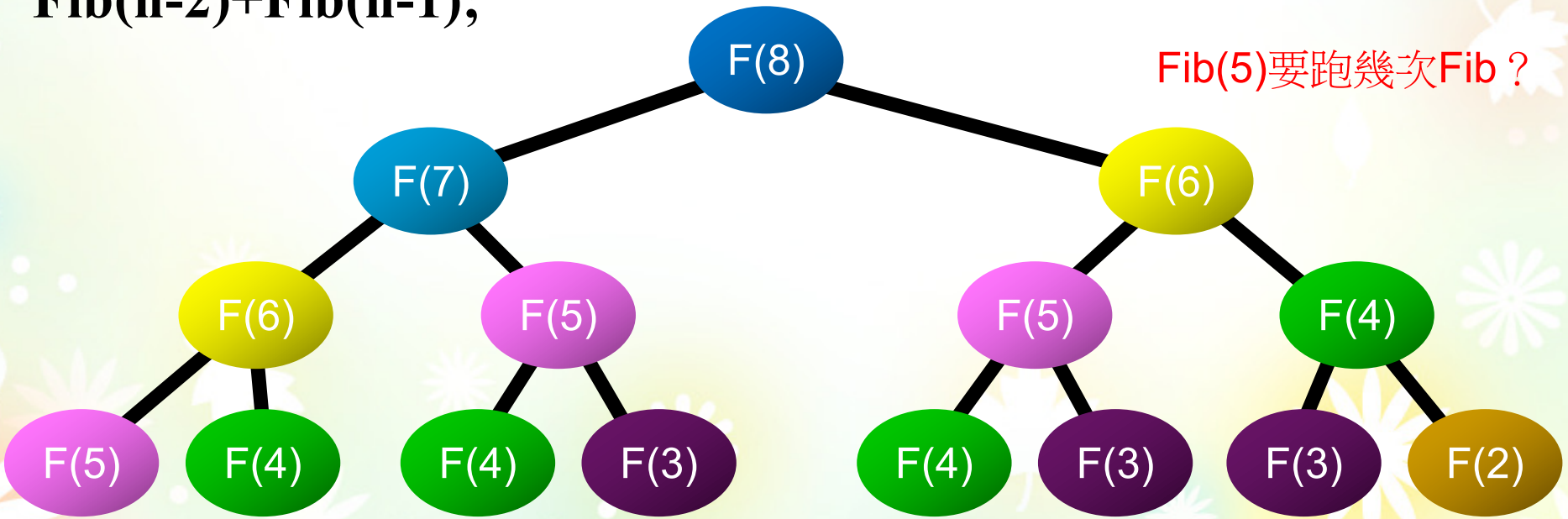
The recursive algorithm

```
int Fib(n)
{
  if n==2||n==1
     return 1;
  else
     Fib(n-2)+Fib(n-1);
}
```

It is very inefficient

It has to take $\Theta(1.6^n)$ time to computer Fib(n) using recursive algorithm

1.6=(1+sqrt(5))/2 (golden ratio)



Fib(5)要跑幾次Fib？

http://stackoverflow.com/questions/360748/computational-complexity-of-fibonacci-sequence

- Why do we want to use recursive functions (or algorithms)?
  - It allow us to express an otherwise complex process in very clear terms.
  - Often the recursive function is easier to understand than its iterative counterpart.

Recursion for BinarySearch

BinarySearch(a,x,0,n-1)

Program 1.11 Recursive Implementation of binary search

```
int BinarySearch( int *a, const int x, const int left, const int right)
{    if(left<right){
     int middle=(left+right)/2;
     if(x<a[middle]) return BinarySearch(a,x,left, middle-1);
     else if (x>a[middle]) return BinarySearch(a,x,middle+1,right);
               return middle;
                }//end of if
               return -1;//not found
}
```

Recursive Permutation Generator

- Example 1.5 [Permutation generator, 排序產生器] Given a set of n>=1 elements, the problem is to print all possible permutation of this set. [{a,b,c}→{(abc),(acb),(bac),(bca),(cab),(cba)}
  - In the case of four elements {a, b, c, d}
  - The answer can be constructed by writing:
    - a followed by all permutations of (b, c, d)
    - b followed by all permutations of (a, c, d)
    - c followed by all permutations of (a, b, d)
    - d followed by all permutations of (a, b, c)
  - " w followed by all permutations of (x, y, z)" is the clue to recursion!
  - It implies that we can resolve problem for a set with n elements if we have an algorithm that works on n-1 elements
  - 有一個演算法可以解決n-1個元素的問題，就可以用來解n個元素的問題

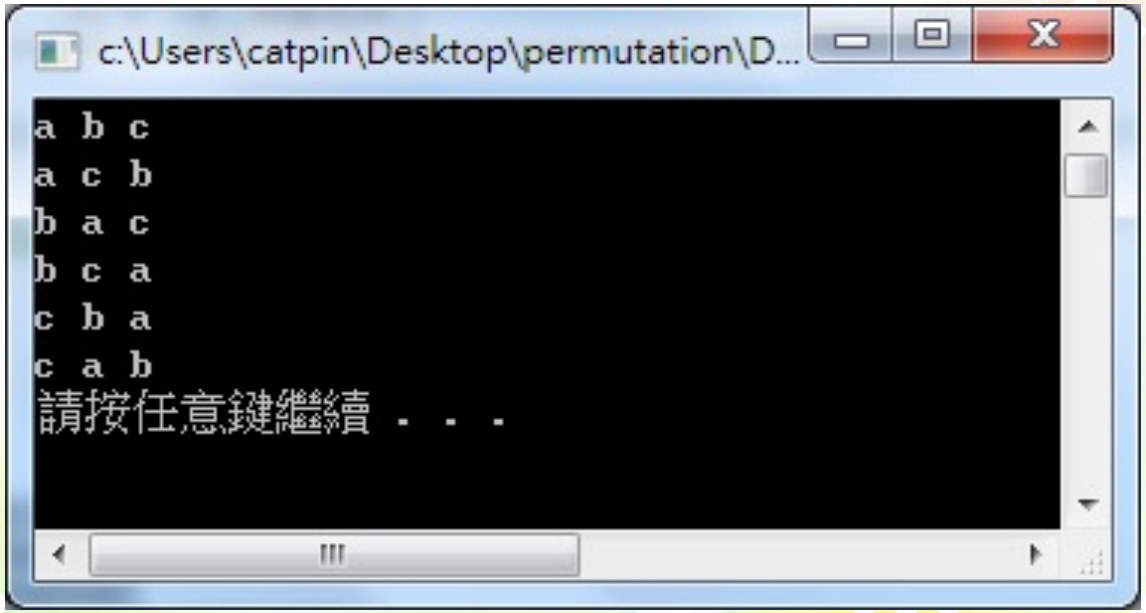
Program 1.12 Recursive pertmutation generation

```
void Perm (char *a, const int k, const int m)
{//generate all the permutation of a[k],….a[m]
    if (k==m){//output permutation
       for (int i=0; i<=m; i++) cout <<a[i]<<“ “;
         cout << endl;}
    else //a[k:m] has more than one permutation. Generate these recursively
       for(int i=k; i<=m; i++)
          swap(a[k],a[i])
          perm(a,k+1,m)
          swap(a[k],a[i]);
         }
}
```

```
#include "stdafx.h"
#include <iostream>
using namespace std;

void Perm (char *a, const int k, const int m)
{//generate all the permutation of a[k],°K.a[m]
     if (k==m){//output permutation
        for (int i=0;i<=m;i++) cout <<a[i]<<" ";
          cout << endl;}
     else //a[k:m] has more than one permutation. Generate these recursively
        for(int i=k;i<=m;i++)
        {
           swap(a[k],a[i]);
           Perm(a,k+1,m);
           swap(a[k],a[i]);
         }
}

int _tmain(int argc, _TCHAR* argv[])
{
    char a[]={'a','b','c'};
    Perm(a,0,2);
    system("pause");
    return 0;
}
```



Performance Analysis and Measurement

- One goal of this book is to develop skills for making evaluative judgments about programs.
- There are many criteria upon which we can judge a program (評論程式碼)
  - Dose it do what we want it to do?
    - 是我們想要做的嗎？
  - Does it work correctly according to the original specifications of the task?
    - 做的是正確根據任務的原始需求嗎？
  - Is there documentation that describes how to use it and how it works?
    - 有任何文件描述如何使用跟程式是如何運行的嗎？
  - Are functions created in such a way that they perform logical subfunctions?
    - 這些函式都有其負責的功能嗎？
  - Is the code readable?
    - 這些程式碼都可以被看的懂嗎？

- Space complexity
  - The amount of memory a program needs to run to completion.
  - 空間複雜度：完成此程式所需要的記憶體大小
- Time complexity
  - The amount of computer time a program needs to run to completion.
  - 時間複雜度：完成此程式所需要花的時間

Space Complexity

- Fixed part
  - Independent of the characteristics (e.g. number, size) of the inputs and outputs.
    - Instruction space 指令空間
    - Space for simple variables, constants 簡單變數 固定大小變數
    - Fixed-size component variables 存放常數空間
- Variable part
  - Component variables whose size is dependent on the particular problem instance. 跟輸入有關（不同輸入可能需要不同的變數變量）
  - Using recursive functions. 遞迴所需的堆疊空間

Space Complexity

- S(P): The space requirement of any program P.

$$S(P) = c + S_P(I)$$

  - c: Fixed Part
  - S_P (I): Variable Part
  - I: Instance characteristics
- Example

```
float abc(float a, float b, float c) {
        return a + b + b*c + (a +b – c)/(a + b) + 4.0 ; }
```

$S_{abc}(I) = 0.$

The space needed by this function abc is independent of the instance characteristics. So, Sp=0;

Space Complexity

- S(P): The space requirement of any program P.

$$S(P) = c + S_P(I)$$

  - c: Fixed Part
  - $S_P(I)$: Variable Part
  - I: Instance characteristics

- 空間複雜度是透過演算法計算時所需的儲存空間
- 包含儲存程式本身指令、常數及變數和輸入資料外，還要儲存對資料操作的輔助單元
- 若輔助空間不會隨著問題的規模(輸入資料)而變化，則空間複雜度為**O(1)**

O(1)就是所謂的常數 沒有O(2)….O(100)

Space Complexity Examples

```
//Iterative function for sum
float sum (float *a, int n) {
        float s = 0 ;
            for (int i = 0; i < n; i++)
                  s =s+a[i] ;
            return s ;
      }
```

The problem instance for this program are characterized by n.

n is passed by value, one word must be allocated for it

Since a is actually the address of the first element in a[], the space needed by it is also one word.→ the space needed by function sum is independent of n and Ssum(n)=0

Space Complexity Examples

```
float rsum (float *a, const int n) {
        if (n <= 0) return 0 ;
        else return (rsum(a, n-1) + a[n-1]) ;
}
```

- It will run n times recursion (n+1 times). 遞迴深度
- Space needed in a recursion in stack (16 bytes, 4 words)
  - float a: 4 bytes
  - const int n: 4 bytes
  - return value: 4 bytes
  - return address: 4 bytes

每一次呼叫rsum都需要宣告出
一個a, 一個n, 傳回值跟傳回位址

- $S_{rsum}(n)=16(n+1)$ bytes$=4(n+1)$ words

Time Complexity

- The time T(P) of program P
  - Compile time 編譯時間
    - Independent to the instance characteristics.
  - Run (or execution) time (T_P) 執行時間
- Obtain an exact formula of T_P is not easy.
  - The true value of T_P (n) for any given n can be obtained only experimentally. 事後統計法
- Program step 程式步驟
  - A syntactically or semantically meaningful segment of a program.

    return a+b+b*c+(a+b-c)/(a+b)+4.0; 事前分析估算法

It could be regarded as a step since its execution time is independent of the instance characteristics

```
int i, sum=0, n=100;
for (i=0;i<=n;i++)
{
sum=sum+i;
}
printf("%d",sum);
```

1+(n+1)+n+1=2n+3

```
int i, sum=0, n=100;
sum=(1+n)*n/2;
printf("%d",sum);
```

1+1+1=3

去掉頭尾
若將迴圈視為一個整體
大概就是n次與1次之間的差距

測量執行時間最可靠的方法，就是計算對執行時間有貢獻的基本操作執行次數

Time Complexity Example

```
//This is a example
float sum (float *a, int n)  { /* 2n + 3*/
    float s = 0 ;
    count++ ;
    int i;
    for (i = 0; i< n; i++) {
        count++ ;
        s += a[i] ;
        count++ ;
    }
    count++ ; /* for the last time of for */
    count++ ; /* for return ;  */
    return s ;
}
```

Comments are not executable statements (step count=0)

Assignment statements (step count=1)

Declarative statements (step count=0)

Iteration statements (step count=n)

Assignment statements (step count=n)

Iteration statements (step count=1)

Jump statements (step count=1)

2n+3

Time Complexity Example

```
//This is a example
float Rsum (float *a, int n)  { /* 2n + 2*/
    count++ ;
    if(n<=0)
        {count++;
         return 0;}
    else
        {count++;
         return  (Rsum(a, n-1)+a[n-1]);
}
```

Comments are not executable statements (step count=0)

Iteration statements (step count=1)
//if

Jump statements (step count=1)
return

Jump statements (step count=1)
return

2 choose 1

$T_{rsum}(n)=$
$2+T_{rsum}(n-1)=$
$2+2+T_{rsum}(n-2)$
……..
$2+2+….+2+T_{rsum}(0)$

2n

$T_{rsum}(0)=2$
$=2n+2$

Program 1.22, 1.23, 1.24, 1.25 Table 1.1,1.2, 1.3
回家自己看一下倒底怎樣計算步驟

- Comparing the step count of previous two examples

  1^{st} : 2n+3 2^{nd}: 2n+2

- The count of second example is less than the first one!
- Could we conclude that the first program is slower than the second one??
- **N O**
- This is so because **a step does not correspond to a define time unit**! Each step of Rsum might take more time than every step of Sum

- 同樣的步驟數目可能因為使用函數不同，速度就不同
- 若牽扯到記憶體搬移也需要時間！

- The step count is useful in that it tell us how the run time for a program changes with changes in the instance characteristics
- From the step count for Sum, we see that if n is doubled, the run time will also double
- If n increase by a factor of 10, we expect the run time to increase by a factor of 10
- → the run time to grow *linearly* in n

Summary of Step

- The number of steps is itself a function of the instance characteristics.
  - 步驟數本身即是實際資料特性的函數
- Any specific instance may have several characteristics.
  - 每一筆實際資料都可能包含許多不同的特性(輸出入個數)
- We need to know exactly which characteristics of the problem instance are to be used.
  - 因此在計算程式步驟前，要瞭解要使用的實際資料特性為何

Three Kinds of Step Counts

- Best-case 最佳步驟數
  - The minimum number of steps that can be executed for the given parameters.
  - 一個程式最少需要執行的步驟數
- Worst-case 最差步驟數
  - The maximum number of steps that can be executed for the given parameters.
  - 一個程式最多需要執行的步驟數
- Average-case 平均步驟數
  - The average number of steps that can be executed with the given parameters.
  - 一個程式平均需要執行的步驟數

Asymptotic Notation 漸近表示法

- Motivation to determine step counts
  - Compare the time complexities of two programs that compute the same function. 比較兩個程式執行效率
  - Predict the growth in run time as the instance characteristics change. 預測實際資料改變時執行時間增加狀況
- Asymptotic notation
  - Determine the **exact step count** is very difficult.
  - x=y;
  - x=y+z+(x/y)+(x*y*z-x/z)

  are regarded as one step

Example: Sum

```
int i, sum=0;n=100;
for( i=1; i<=n; n++)
{
sum=sum+i;
}
cout<< “The total sum=”<<sum<<endl;
```

執行1次

執行n+1次

執行n次

執行1次

執行2n+3次

```
int i, sum=0;n=100;

sum=n*(n+1)/2;

cout<<“The total sum=”<<sum<<endl;
```

執行1次

執行1次

執行1次

執行3次

Better!!!

```
int i, j, x=0;sum=0;n=100;
for( i=1; i<=n; n++)
{
    for (j=1;j<=n;j++)
     {
    x++;
    sum=sum+x;
     }
}
cout<< “The total sum=”<<sum<<endl;
```

執行1次

執行n*n次
(若忽略迴圈頭尾的次數)

執行1次

執行n^2+3次

When n=100 其執行次數多於前面兩個演算法
所以這個演算法的執行時間將隨n的增加而遠大於上述兩個
100*100+3>> 100+3>>3 (**3看起來好像不太重要**)

→測量執行時間最可靠的方法，就是**計算執行時間有貢獻之基本操作執行次數**

Counting Primitive Operations

- By inspecting the pseudocode, we can determine the **sum** of primitive operations executed by an algorithm, as a function of the input size

```
int i, sum=0;n=100;
for( i=1; i<=n; n++)
{
sum=sum+i;
}
cout<< “The total sum=”<<sum<<endl;
```

執行1次
執行n+1次
執行n次
執行1次

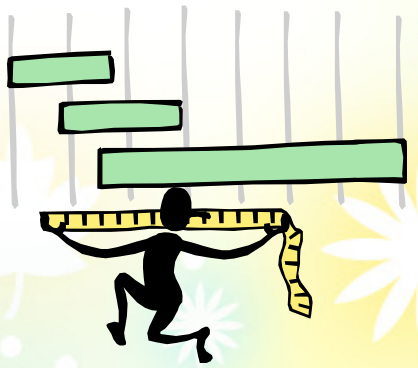
執行2n+3次

Estimating Running Time

- Algorithm ***SUM*** executes $2n+3$ primitive operations in the worst case. Define:
  - a = Time taken by the **fastest primitive operation** 最快的操作運算子
  - b = Time taken by the **slowest primitive operation** 最慢的操作運算子
- Let $T(n)$ be worst-case time of ***SUM***. Then

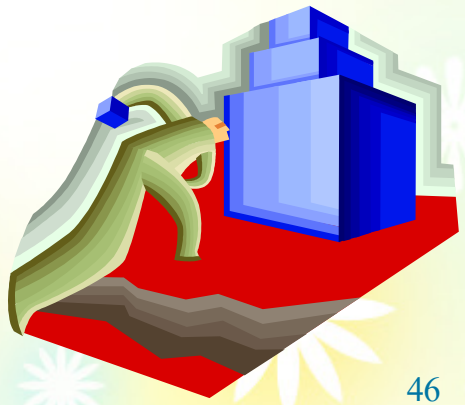
$$a(2n+3) \le T(n) \le b(2n+3)$$

- Hence, the running time $T(n)$ is bounded by two linear functions



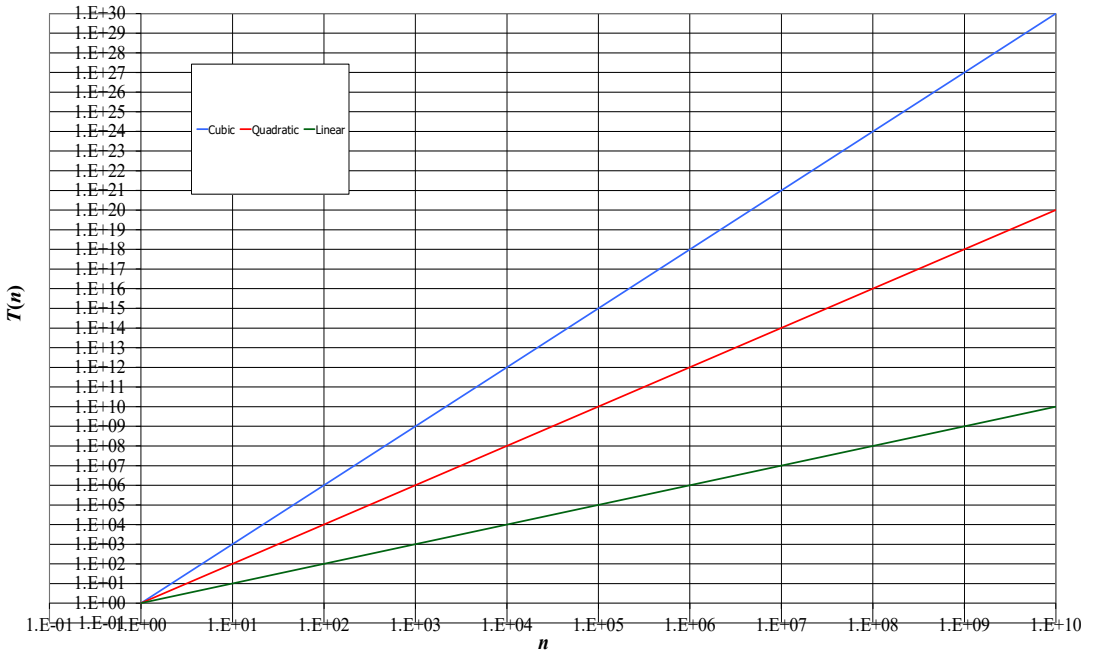
Growth Rate of Running Time

- Changing the hardware/software environment
  - Affects $T(n)$ by a constant factor, but
  - Does not alter **the growth rate of $T(n)$**
- The linear growth rate of the running time $T(n)$ is an intrinsic property of algorithm ***SUM***



Growth Rates

- Growth rates of functions:
  - Linear ≈ n
  - Quadratic ≈ n^2
  - Cubic ≈ n^3
- In a log-log chart, th slope of the line corresponds to the gr rate of the function



- For most situations, it is adequate to able to make a statement like
  - $c_1n^2 \leqq t_p(n) \leqq c_2n^2$

  or $t_Q(n,m)=c_1n+c_2m$
    - c1,c2 are nonnegative constants

- 如果是 $c_1n^2+c_2n$ & c_3n 比大小？？
- When n→large value, c_3n will be faster than $c_1n^2+c_2n$
- When n→small value, either program could be faster, depending on c_1, c_2 and c_3

If $c_1=1$, $c_2=2$ and $c_3=100$
$c_1n^2+c_2n \leqq c_3n$ for $n \leqq 98$

If $c_1=1$, $c_2=2$ and $c_3=100$
$c_1n^2+c_2n > c_3n$ for $n > 98$

If $c_1=1$, $c_2=2$ and $c_3=1000$
$c_1n^2+c_2n \leqq c_3n$ for $n \leqq 998$

- No matter what the value of c_1, c_2 and c_3, there will be an n beyond which the program with complexity c_3n will be faster than $c_1n^2+c_2n$
- 不管c_1, c_2 and c_3的數值為何，一定存在一個足夠大的n，使得c_3n的程式效率比$c_1n^2+c_2n$ 快
- The value of n will be called the *break-even point*
  - *The program have to be run on a computer for determination of the break-even point.*
- To know there is a break-event point, it is adequate to know that one program has complexity $c_1n^2+c_2n$ and the other c_3n for some constants: c_1, c_2 and c_3.
- That is little advantage in determining the exact value

次數	演算法A(2n+3)	演算法A'(2n)	演算法B(3n+1)	演算法B'(3n)
n=1	5	2	4	3
n=2	7	4	7	6
n=3	9	6	10	9
n=10	23	20	31	30
N=100	203	200	301	300

n=1 演算法A效率不如演算法B

n=2 效率相同

n>2 演算法A效率開始優於演算法B

結論：演算法A總體效率比演算法B好

函式的漸進增長

給定兩個函式f(n)跟g(n)，如果存在一個整數N，且所有n>N，f(n)總是比g(n)大，則f(n)的增長會逐漸快於g(n)

隨著n的增加，演算法次數的常數項+3 or +1其實都不影響最終的演算法，因此可以忽略這些加法常數